

Dokumentacja wstępna projektu

Przedmiot: Algorytmy ewolucyjne

Temat: Modyfikacja algorytmu DE/rand/1/bin z adaptacją parametru F metodami MSR i PSR

Autorzy: Mykhailo Shamrai (329923), Grzegorz Prasek (327394)

1. Opis problemu i sposób rozwiązania

Celem projektu jest modyfikacja algorytmu ewolucji różnicowej w wariantcie **rand/1/bin** poprzez zastąpienie stałego parametru mutacji F mechanizmem adaptacyjnym opartym na regułach sukcesu: *Median Success Rule* (MSR) oraz *Population Success Rule* (PSR). W klasycznym DE parametr F skaluje wektor różnicowy i w praktyce w dużym stopniu decyduje o kompromisie między eksploracją i eksploatacją przestrzeni poszukiwań. Z tego powodu jego odpowiednie strojenie ma istotny wpływ na jakość działania algorytmu.

Punktem wyjścia będzie standardowy algorytm DE/rand/1/bin, w którym dla każdego osobnika generowany jest wektor mutant:

$$v_i = x_{r_1} + F_t(x_{r_2} - x_{r_3}),$$

następnie wykonywane jest krzyżowanie binarne z rodzicem i ostatecznie wybierany lepszy osobnik z początkowego oraz tego po modyfikacji. W proponowanych modyfikacjach wartość F_t nie będzie stała, lecz aktualizowana po każdej iteracji na podstawie obserwowanego sukcesu bieżącej populacji potomków. Reguły MSR i PSR pochodzą oryginalnie ze strategii ewolucyjnych, gdzie służą do strojenia rozmiaru kroku mutacji; w projekcie zaadaptujemy je do strojenia parametru F w DE.

W wariantcie **MSR** sukces będzie mierzony liczbą potomków, których wartość funkcji celu jest lepsza niż mediana wartości funkcji osobników z populacji poprzedniej. Gdy udział takich osobników przekracza docelowy poziom sukcesu, parametr F zostaje zwiększony, w przeciwnym razie zmniejszony. W wariantcie **PSR** populacje z dwóch kolejnych iteracji łączone są w jeden zbiór i sortowane; sygnał sukcesu wyznaczany jest na podstawie różnicy średnich rang osobników starej i nowej populacji. W odróżnieniu od MSR, reguła PSR korzysta wyłącznie z uporządkowania osobników, dzięki czemu jest odporna na skalę wartości funkcji celu.

W obu wariantach aktualizacja F będzie prowadzona multiplikatywnie, z tłumieniem oraz ograniczeniem do zakresu $[F_{\min}, F_{\max}]$ w celu uniknięcia degeneracji. Jedna wartość F_t będzie obowiązywać dla całej populacji w danej iteracji.

Planowane są trzy wersje algorytmu:

- bazowy DE/rand/1/bin ze stałym F ,
- DE-MSR – adaptacja F zgodnie z regułą MSR,
- DE-PSR – adaptacja F zgodnie z regułą PSR.

W celu zachowania porównywalności pozostałe elementy algorytmu będą utrzymywane bez zmian, tj. ta sama strategia mutacji, identyczny rozmiar populacji oraz identyczna metoda obsługi. Ewentualne parametry pomocnicze reguł adaptacyjnych (docelowy poziom sukcesu, współczynnik tłumienia, zakres F) jeszcze są do ustalenia.

2. Planowane eksperymenty numeryczne

Badania zostaną przeprowadzone na benchmarku **CEC 2017** dla wymiarów:

$$D \in \{10, 30\}.$$

Plan eksperymentów obejmuje:

- uruchomienie każdej z trzech wersji algorytmu na wszystkich funkcjach testowych benchmarku CEC 2017,
- wykonanie wielu niezależnych uruchomień (docelowo zgodnie z protokołem CEC, tj. rzędu kilkudziesięciu powtórzeń na funkcję),
- przyjęcie wspólnego budżetu obliczeniowego wyrażonego liczbą wywołań funkcji celu (standardowo $10^4 \cdot D$),
- porównanie jakości końcowych rozwiązań oraz przebiegów zbieżności.

Podstawową miarą jakości będzie wartość błędu końcowego względem znanego optimum funkcji testowej. Dla każdego wariantu planujemy raportować co najmniej: średnią, medianę, odchylenie standardowe oraz najlepszą i najgorszą uzyskaną wartość. Dodatkowo planujemy zastosować testy statystyczne do porównania metod (np. test Wilcoxona dla par algorytmów oraz ranking zbiorczy między wszystkimi metodami).

Poza porównaniem jakości końcowej chcemy przeanalizować sam mechanizm adaptacji, w szczególności przebieg zmian parametru F w czasie oraz zależność jego zachowania od typu funkcji testowej (unimodalne, multimodalne, hybrydowe, kompozycyjne). Pozwoli to ocenić, czy reguły MSR i PSR prowadzą do stabilnej oraz użytecznej adaptacji w algorytmie DE.

3. Wybór technologii

Projekt planujemy zrealizować w języku **Python**. Python jest wygodny do szybkiej implementacji, prowadzenia dużej liczby eksperymentów numerycznych oraz późniejszej analizy wyników.

Wstępnie zakładamy użycie następujących narzędzi:

- **NumPy** – operacje numeryczne i obsługa populacji,
- **SciPy** – testy statystyczne,
- **matplotlib** – wizualizacje,
- implementacja funkcji testowych zgodna z benchmarkiem **CEC 2017**.